E Waste Compute Grid Project Guide

Capabilities, implementation changes, setup and hackathon operation

Prepared for the project team on 7 October 2026

1 What the project does

E-Waste Compute Grid lets a team use older computers as workers for independent computing tasks. A central server divides a submitted job into smaller tasks. Available workers request tasks, execute the supported workload, upload results and report timing. The dashboard shows the machines, job progress and estimated energy consumed by successful executions.

The current main workload is **image classification using MobileNetV2 and ONNX Runtime on a CPU**. For each image, it returns a predicted ImageNet class and a confidence value. A second workload produces deterministic demo results so the team can demonstrate the distributed pipeline without relying on meaningful image predictions.

Current readiness: the local API, PostgreSQL queue, Python workers, input transfers, real inference, result uploads and aggregation were exercised together. All 38 automated checks passed across the backend, worker and workload suites. Both application builds and frontend lint passed. These results verify the tested local scenarios; they do not establish that every deployment or hardware configuration will work without further testing.

Docker execution on physical workers, the complete Docker Compose stack, browser interactions and live AWS storage/deployment have not been verified in this session. Docker Desktop failed while initializing its local inference service. The existing host database credentials were also rejected, so validation used a separate PostgreSQL instance and isolated schemas.

Read this guide in this order

- For a quick demo, use sections 6 and 8.
- For development without Docker, use section 7.
- For multiple physical machines, use section 9.
- For implementation details and changed code, use sections 3 to 5.
- For API integration and verification, use sections 11 and 12.
- For troubleshooting, limitations and the presentation, use sections 13 and 14.

A simple example

Submit 20 images with 5 images per task. The server creates 4 tasks. If two compatible workers are available, each can process one task at a time and request another when finished. The job ends when every task is completed or permanently failed. Its downloadable JSON combines the successful predictions and any errors.

This is batch distribution across machines. It does not join several CPUs into one shared processor or split a single neural-network inference across workers.

2 Features and their current status

Feature	What the implementation provides	Evidence or limit
Worker enrollment	Shared enrollment token, persistent node identity and scoped node secret	Tested through HTTP and Python workers
Hardware reporting	CPU count, fractional RAM, OS, hostname, supported workloads and estimated baseline watts	Tested with actual worker registration
Task scheduling	Worker polling, workload capability and RAM filters, oldest compatible task first	Concurrent assignment tested
Job creation	Validated inputs, configurable chunk size and idempotency key	Invalid sizes and repeat requests tested
Image uploads	Individual JPG, JPEG, PNG, WebP or BMP files	Local upload/download exercised
Image classification	CPU inference using MobileNetV2 FP32	Real inference and corrupt-image isolation tested
Pipeline demo	Deterministic artificial predictions from text items	Two real workers tested concurrently
Recovery	Leases, heartbeat extension, bounded retries and stale-attempt rejection	Timeout, duplicates and retry exhaustion tested
Results	Job JSON containing predictions, errors, counts and execution metrics	Completed and partial jobs tested
Dashboard	Connection form, real polling, node list, submission form and results button	Build and lint passed; browser flow unverified
Energy	Estimated watt-hours from baseline power and successful compute duration	Formula and metering helper tests passed
S3 adapter	AWS SDK upload, existence checks, signed URLs and result reads	Implemented; live AWS not tested
Container deployment	Compose definitions for database, API, frontend and two simulation workers	Configuration added; stack not run successfully here

What image inference returns

Each prediction includes the input filename, class ID, human-readable class name and confidence. The model uses approximately 1,000 ImageNet categories. A confidence value is model output, not a guarantee that the label is correct.

The bundled model was inspected directly: its weights are floating point and it has no quantization operators. The filename was corrected from `mobilenetv2-int8.onnx` to `mobilenetv2-12.onnx`. It is approximately 13.3 MiB. No new model training was performed.

Capabilities outside the active pipeline

The repository also contains hardware benchmarking, weighted worker ranking, optional power-meter sampling, an environmental calculator and a model artifact helper. These are supporting utilities. The running scheduler uses capability and RAM filtering with FIFO ordering; it does not call the separate Python ranking engine. The dashboard does not display the environmental helper's hypothetical carbon or cloud savings.

3 How the system works

The components

Component	Responsibility	Main files
React frontend	Submit jobs and display API state	frontend/src/App.tsx and services/api.ts
Express control plane	Authenticate, validate, assign and complete tasks	control-plane/src/app.ts
PostgreSQL and Prisma	Persist nodes, jobs, tasks and metrics; coordinate concurrent workers	control-plane/prisma/schema.prisma
Storage adapter	Store inputs and output JSON; create expiring transfer URLs	control-plane/src/storage.ts
Python worker	Register, heartbeat, request work, execute and report	worker/agent.py and task_executor.py
Workloads	Run image classification or the demo	workloads/ai-inference and dummy-workload

One job from beginning to end

1. A worker checks its execution environment, reports its hardware and enrolls using the enrollment token. Its returned credentials are saved locally.

2. The user connects to the dashboard with the admin token. For image classification, the dashboard uploads the selected files and receives storage keys.

3. The user submits the name, workload, inputs and chunk size. The backend validates the request and creates the job and tasks together in a database transaction.

4. A worker requests work. The backend locks an eligible queued task, records the worker and creates a unique lease token for that attempt.

5. The worker stages only that task's inputs. Its heartbeat runs separately while downloading, computing and reporting results.

6. The worker executes a fixed supported workload. Docker is the default on physical workers; explicit native mode executes repository-owned scripts for development.

7. The worker asks for a fresh upload URL and uploads `result.json`. It then submits the task ID, lease token, output key and compute duration.

8. The backend validates the output, records the accepted result and metric once, and reconciles the job counts and final state.

9. The dashboard polls approximately every three seconds. Terminal jobs expose a JSON results download.

Data and trust boundaries

The database is the source of truth for assignments and accepted results. File storage holds uploaded inputs and attempt-specific outputs. Worker credentials are never returned by dashboard synchronization. Workers select images from a local allowlist; a dashboard user cannot submit arbitrary Python code or Docker image names.

The MVP assumes trusted team computers. Authentication and container restrictions improve control, but this is not a public, adversarial volunteer-computing platform.

4 Code changes in the control plane

API and application structure

The original backend placed most logic in one TypeScript file and retained an older JavaScript implementation. The stale JavaScript source was removed. The backend is now separated into `app.ts` for HTTP/application behavior, `index.ts` for startup and shutdown, `config.ts` for configuration, `validation.ts` for input checks, and `storage.ts` for file storage.

Registration and heartbeats now use the same Bearer-token convention as the worker. Enrollment is repeatable using a registration key, avoiding duplicate nodes when an enrollment response is lost. Worker task endpoints validate the node secret, and dashboard/job endpoints require the admin token.

Database changes

Node records now support a display name, fractional RAM, floating-point baseline watts and a unique registration key. Job records track failed tasks and an optional unique submission key with its request fingerprint. Tasks track an attempt token, assignment time, creation time, last error and a related metric. A unique metric task ID prevents more than one accepted metric per task. Queue, lease and relationship indexes were added.

These schema changes require `prisma db push` for the demo database. For a deployed database containing valuable data, review schema changes and use a managed migration and backup process instead of treating the demo startup command as a production migration system.

Missing behavior completed

• Added input upload, result upload URL, task failure, health and job results endpoints.

• Added local disk storage and an optional AWS S3 adapter.

• Added job progression to IN_PROGRESS and terminal COMPLETED, PARTIAL_SUCCESS or FAILED states.

• Added output validation before accepting completion and server-side computation of estimated energy.

• Added heartbeat lease renewal with a maximum attempt lifetime.

• Made repeated completion and failure requests safe for the same attempt.

• Added a guarded recovery sweep and graceful server shutdown.

Concurrency corrections

Task assignment uses PostgreSQL row locking with `SKIP LOCKED`. Node-level locking prevents a worker from receiving multiple active tasks through simultaneous polls. A repeated poll returns the existing live assignment so a lost HTTP response does not strand it.

Completion and timeout handling lock the task before changing its state. Job counters are reconciled while holding a job lock. The real concurrency tests exposed a foreign-key lock upgrade deadlock; node and job locks were changed to `FOR NO KEY UPDATE` so metric insertion can coexist with the required serialization.

Build and dependency changes

Added working dev, build, start, database and test commands. Prisma Client moved to runtime dependencies. AWS SDK storage packages and the backend lockfile were added. The incompatible Prisma skill-sync configuration was removed. Database client generation is part of the build.

5 Changes in the worker frontend and workloads

Python worker

The worker now reads `task_id`, `lease_token` and per-item input metadata matching the backend. It sends Bearer authentication and `x-node-id`, requests the correct upload endpoint, uploads JSON and completes the corresponding attempt. Registration reports a workload list rather than an encoded hardware JSON string.

Credentials are saved atomically. An operating-system file lock prevents two agents sharing one identity directory. Heartbeats run independently, and lost leases trigger cancellation. Upload/completion delivery retries do not rerun successful compute. A failure always clears the active task state. Worker restart removes only containers labeled for that worker identity.

Docker execution uses fixed configured images, read-only inputs, a read-only container filesystem, disabled networking, dropped capabilities, CPU/RAM/process limits and a timeout. Native development mode also has a timeout and kills the child process when cancelled. Input filenames are checked for traversal and duplicates. Transfers and result files have size limits.

Frontend

The active dashboard and API client were consolidated. Disconnected alternative pages, layout components and their incompatible API shapes were removed. The dashboard now supports an admin-token connection, genuine loading/error states, file uploads, demo jobs, chunk sizing, accurate task counts and result downloads.

Mock fallback data and simulated successful submissions were removed. Failed submissions remain visible. The API base URL is configurable, with a local Vite proxy by default. Poll requests do not overlap; pending requests are aborted when the component or connection changes.

Workload utilities

The result aggregator no longer refers to an undefined previous prediction when a failed task appears first. It no longer duplicates errors, and it reports failed-only or mixed results appropriately. Unknown workload types no longer silently fall back to a dummy success. Empty image directories fail explicitly; malformed or empty result output is rejected by the task runner.

Image loading now handles EXIF orientation and closes files properly. Inference thread count is bounded. The power-meter helper includes the interval boundaries so short tasks record energy. Invalid energy inputs are rejected. The cloud comparison is labeled as a hypothetical equal-runtime estimate, not measured savings. Model checksum failure now stops the downloader.

Infrastructure and checks

Added Dockerfiles for the frontend and simulation worker, refreshed Compose, and added environment examples and ignore rules. Added backend validation/integration tests, worker tests, additional workload regressions and a GitHub Actions workflow. The workflow is present in the repository; no remote CI run was performed during this work.

6 Start the complete demo with Docker

This is the intended one-command demo setup. Its full container execution still needs verification on a machine with a working Docker engine. The two included workers are containerized simulations running the bundled Python scripts in native mode inside their worker containers; physical workers use separate workload containers as described in section 9.

Requirements

• Docker Engine/Desktop running Linux containers, with Compose available.

• Sufficient disk space and internet access for the first image builds.

• Free host ports 5173 and 8080; PostgreSQL is bound to localhost port 55432.

• The complete repository, including the bundled model and ImageNet label file.

From a terminal in the project root, run:

```
docker version
docker compose -f infra/docker-compose.yml up --build -d
docker compose -f infra/docker-compose.yml ps
```

Check that the database and API are healthy and both workers are running. Open `http://localhost:5173`. The health endpoint is `http://localhost:8080/api/health`.

For the unchanged local demo defaults, connect with admin token `local-admin-change-me`. Worker enrollment uses `local-enrollment-change-me`. These are intentionally public development defaults, not production secrets. If the team overrides ADMIN_TOKEN, enter that configured value instead.

Services started

Service	Purpose	Host access
db	Persistent PostgreSQL database	localhost:55432
control-plane	API, queue and recovery loop	localhost:8080
frontend	Dashboard and API proxy	localhost:5173
worker-1 and worker-2	Two independent simulation agents	No inbound ports needed

The Compose volumes preserve database state, object storage and worker credentials between ordinary restarts. Each simulation worker has its own identity.

Logs and shutdown

```
docker compose -f infra/docker-compose.yml logs --tail=100
docker compose -f infra/docker-compose.yml down
```

Ordinary `down` keeps named volumes. Adding `-v` deletes the stored demo data and worker identities; do not use it when you intend to preserve jobs and results. This repository does not provide a user-facing delete/reset feature.

7 Run locally without Docker

Use this development path when Docker is unavailable. The integrated local tests used a real PostgreSQL server and Python native execution. Install Node.js 22 or newer, Python 3.12, and a running PostgreSQL server. Create a project database with credentials you control.

Prepare Python from the project root

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r worker/requirements.txt
.venv\Scripts\python.exe -m pip install -r workloads/ai-inference/requirements.txt
```

Use the project virtual environment rather than changing system Python packages. On Linux/macOS, use `.venv/bin/python` in place of `.venv\Scripts\python.exe`.

Start the API in terminal one

```
cd control-plane
npm ci
```

Create or edit `control-plane/.env` using `.env.example` as a reference. Set DATABASE_URL to your working database. If using only the Compose database, its host port is 55432. Preserve any existing settings you need. The old `.env` in this workspace was not overwritten; its original credentials failed authentication during validation.

```
npm run db:push
npm run build
npm start
```

Run these commands from `control-plane` so its environment file is loaded. Confirm `http://localhost:8080/api/health` returns status `ok`.

Start the worker in terminal two from the project root

```
$env:EXECUTION_MODE='native'
$env:CONTROL_PLANE_URL='http://localhost:8080/api'
$env:ENROLLMENT_TOKEN='local-enrollment-change-me'
.venv\Scripts\python.exe worker/agent.py
```

Use the enrollment token configured on your API if different. Native mode runs only the supplied workload scripts. It is a development fallback and does not provide the per-task Docker isolation of the normal physical-worker path.

Start the dashboard in terminal three

```
cd frontend
npm ci
npm run dev
```

Open `http://localhost:5173` and enter the admin token. Keep all three terminals running; Ctrl+C stops the corresponding process. A stopped worker's active task is recovered after its lease expires.

8 Use the dashboard and understand results

Run the first pipeline demo

1. Connect with the API's admin token. Wait for the worker list to show at least one online or idle node.
2. Give the job a name such as `First grid demo`.
3. Choose **Pipeline demo**. Enter 20 demo items and 5 items per task.
4. Submit. Expect 4 tasks, not 20 tasks.
5. Watch the job move from QUEUED to IN PROGRESS and then COMPLETED.
6. Download results. Confirm 20 artificial predictions and the recorded metrics.

The demo generates deterministic labels from filenames. Its predictions are not AI image classifications; its purpose is to exercise distribution, progress and result collection.

Run actual image classification

1. Choose **Image classification** and select JPG, JPEG, PNG, WebP or BMP images.
2. Choose a chunk size between 1 and 100. Start with 5 for a small demonstration.
3. Submit and wait for a compatible worker advertising `image_inference` with at least 2 GB reported RAM.
4. Download the terminal job result and inspect the predicted classes, confidence values and errors.

Each file must be nonempty and at most 20 MiB. The API accepts up to 10,000 input items per job. The worker result JSON is limited to 5 MiB. Use genuine photographs for meaningful predictions; some bundled sample files may be placeholders and therefore produce unhelpful labels.

Interpret the job states

State	Meaning
QUEUED	Tasks exist and are waiting for compatible workers
IN_PROGRESS	At least one assignment has started and unfinished tasks remain
COMPLETED	Every task completed without recorded item failures
PARTIAL_SUCCESS	Some useful results exist, but tasks or individual images failed
FAILED	All tasks ended in permanent failure

PENDING remains a schema default, but normal API job creation creates tasks and returns QUEUED in one transaction. Task states in the active implementation are QUEUED, ASSIGNED, COMPLETED and FAILED. Timeout is a recovery event, not a separately persisted task state.

Read the results JSON

The download contains job identity and status, total/completed/failed task counts, item counts, predictions, errors and metrics. Predictions and errors include their task ID. Failed tasks include the last reported error. The backend assembles downloads from accepted database results, so duplicate completion requests do not inflate the output.

Estimated energy is baseline watts multiplied by successful compute duration in hours. For example, 45 W over 10 seconds is 0.125 Wh. It excludes failed attempts, network transfers and full-system idle energy. It is consumption, not proof of energy saved or carbon-negative operation.

9 Add physical worker machines

Each worker needs the repository's worker and workload code, Python dependencies, a working Linux-container Docker engine and outbound access to the control plane and file storage. The controller and worker can be on different computers. They must use a reachable API address, not each machine's own localhost.

Build the allowed workload images on each worker

From the project root:

```
docker build -t ewaste/dummy-workload:v1 workloads/dummy-workload
docker build -t ewaste/ai-workload:v1 workloads/ai-inference
```

The image-classification container bakes in the model and labels. The worker verifies both configured images exist at startup. It does not automatically publish or pull your images from a registry.

Configure and run a worker

Create `worker/.env` from the example, using your controller's actual address:

```
CONTROL_PLANE_URL=http://YOUR_CONTROLLER_IP:8080/api
ENROLLMENT_TOKEN=YOUR_CONFIGURED_ENROLLMENT_TOKEN
EXECUTION_MODE=docker
WORKER_NAME=Lab laptop 1
```

Use HTTPS for a public deployment. Then run:

```
.venv\Scripts\python.exe worker/agent.py
```

Confirm the worker appears in the dashboard. Each agent handles one task at a time, then requests another. Multiple compatible workers allow separate chunks to run concurrently.

Multiple agents on one host

Use different DATA_DIR and TEMP_EXEC_DIR values for every agent. Copying the same credential directory to another active worker is not supported. For development simulation, terminal two could set:

```
$env:DATA_DIR='./worker/.worker_data_second'
$env:TEMP_EXEC_DIR='./worker/.task_exec_second'
$env:WORKER_NAME='Second local worker'
```

Keep the directories outside any input dataset. Restarting an agent with the same identity directory retains its node ID. A repeated assignment after a lost polling response uses the same attempt until the lease expires.

Hardware compatibility

CPU inference is implemented. GPU scheduling, GPU inference and distributed model training are not implemented. Raspberry Pi and other ARM machines need compatible base images and ONNX Runtime wheels; this project was not validated on ARM hardware. Treat a real mixed-hardware demo as a separate acceptance test before presenting it.

10 Configuration storage and deployment

Main API settings

Setting	Purpose and development default
DATABASE_URL	PostgreSQL connection string; must point to a working database
PORT	API port; 8080
ADMIN_TOKEN	Dashboard and job access; local-admin-change-me
ENROLLMENT_TOKEN	Worker enrollment; local-enrollment-change-me
STORAGE_SECRET	Signs local storage URLs; replace outside development
STORAGE_DRIVER	local or s3; local
STORAGE_DIR	Local object directory; .storage relative to backend working directory
PUBLIC_API_URL	URL prefix for signed local transfers; /api by default
CORS_ORIGINS	Allowed browser origins; localhost ports 5173 and 4173
TASK_LEASE_MS	Renewable lease length; 300000 ms
MAX_ATTEMPT_MS	Maximum attempt lifetime; 900000 ms
SWEEP_INTERVAL_MS	Recovery checks; 10000 ms
NODE_OFFLINE_MS	Heartbeat staleness threshold; 90000 ms

Worker defaults are a 15-second heartbeat, polling backoff from 2 to 30 seconds with jitter, and a 600-second execution timeout. The server can request a shorter heartbeat interval. Docker limits default to 2 GB RAM and 2 CPU equivalents and can be changed with WORKLOAD_MEMORY_LIMIT and WORKLOAD_CPUS. Keep the worker timeout below the maximum attempt lifetime to leave time for transfers and reporting.

Local file storage

Inputs use generated keys under `inputs/`. Outputs use job, task and attempt identifiers under `outputs/`. Local URLs expire after one hour and are signed for a specific operation and key. Relative local URLs are resolved against each worker's configured control-plane URL, allowing local and LAN workers to use the same API contract.

If you set an absolute PUBLIC_API_URL, every worker must be able to reach it. Do not advertise `localhost` to workers on other machines. Local storage requires persistent disk and is intended for a single API instance unless a shared filesystem is explicitly configured.

AWS S3 mode

Set STORAGE_DRIVER to `s3`, S3_BUCKET to your bucket and AWS_REGION to its region. Supply AWS credentials through the SDK's normal environment or IAM role mechanism. The API needs object read/write/existence access for the input/output prefixes. Workers receive signed URLs; they do not need AWS credentials for dataset transfers.

S3_ENDPOINT and S3_FORCE_PATH_STYLE support a compatible endpoint. Live AWS permissions, regional deployment, storage lifecycle and recovery were not tested. There is no infrastructure-as-code deployment for App Runner, ECS, RDS, Amplify or CloudFront in this repository.

NODE_ENV=production requires nondefault secrets of at least 24 characters and an HTTPS PUBLIC_API_URL. The application itself does not terminate TLS; configure your deployment proxy/load balancer. Keep the demo Vite preview server and development database setup separate from production deployment decisions.

11 API reference and data contracts

All routes below are relative to the control-plane host. Admin routes use `Authorization: Bearer ADMIN_TOKEN`. Worker routes use `Authorization: Bearer NODE_SECRET` and `x-node-id: NODE_ID`. Enrollment uses the enrollment token. Storage transfer URLs carry their own signatures.

Method and route	Access	Purpose
GET /api/health	Public	Check API database connectivity
POST /api/nodes/register	Enrollment	Create or recover worker identity
POST /api/nodes/:id/heartbeat	Worker	Report liveness and extend a matching live lease
POST /api/inputs?name=FILE	Admin	Upload one raw image and receive its key
POST /api/jobs	Admin	Validate and split a job; requires Idempotency-Key
GET /api/tasks/request	Worker	Return an assignment or HTTP 204
GET /api/tasks/:id/upload-url	Worker	Fresh result upload URL; requires x-lease-token
POST /api/tasks/:id/complete	Worker	Accept result and metrics for the current attempt
POST /api/tasks/:id/fail	Worker	Retry or permanently fail the current attempt
GET /api/dashboard/sync	Admin	Node list, recent 50 jobs and aggregate metrics
GET /api/jobs/:id/results	Admin	Download assembled job JSON
GET or PUT /api/storage	Signed URL	Local storage transfers when local mode is enabled

Demo job body

```
{
  "name": "First grid demo",
  "workload_type": "dummy",
  "items": ["one", "two", "three", "four"],
  "chunk_size": 2,
  "min_ram_gb": 0.25
}
```

Send a unique Idempotency-Key header. Retrying the same body with the same key returns the same job. Reusing the key with different content returns HTTP 409. Image jobs use `image_inference`, normally min_ram_gb 2, and an array of `{ "key": "inputs/...", "name": "photo.jpg" }` objects obtained from uploads.

Assignment and completion fields

Assignments include task_id, job_id, lease_token, lease_expires_at, attempt_deadline, retry_count and payload. The payload contains workload_type, min_ram_gb and items. Each image item has a download_url; each dummy item has content.

Completion contains lease_token, execution_time_ms and result_key. The result key must belong to that attempt and the output must already exist. Failure contains lease_token, error and a Boolean retryable flag. Heartbeats contain state, active_task_id and lease_token; the response action can be NONE or CANCEL_TASK.

Use `docs/api_contracts.md` alongside the code for exact examples. IDs and secrets from one API instance are not interchangeable with another database or server.

12 Failure protection and test evidence

Failure or edge case	Implemented behavior
Two workers poll together	Row locks and SKIP LOCKED prevent shared assignment
One worker repeats a poll	Existing active assignment is returned
Task completion is retried	Same attempt is accepted once; metrics and counts are not duplicated
Worker disappears	Node becomes OFFLINE; expired tasks are requeued by the sweep
Worker reconnects after reassignment	Old lease token is rejected, including on the same node
Worker is alive but task hangs	Execution timeout and maximum attempt lifetime bound the attempt
Task fails repeatedly	Initial attempt plus up to 3 retries, then permanent failure
Some images are corrupt	Successful images are retained; job can finish with partial success
All tasks fail	Job reaches FAILED rather than staying unfinished forever
Invalid chunk size	Request is rejected before task-generation loops run
Result upload is absent or malformed	Completion is rejected
Connection fails during reporting	Worker retries delivery; server completion is idempotent
Unsafe input filenames	Worker rejects traversal and duplicate names
No suitable worker exists	Job stays QUEUED until a capable worker becomes available

Verified results on 7 October 2026

Check	Result
Backend validation and PostgreSQL/HTTP integration	14 passed, 0 skipped
Worker regression tests	7 passed
Workload, fault-tolerance, scheduler and energy tests	17 passed
Backend TypeScript build	Passed
Frontend TypeScript/Vite build	Passed
Frontend lint	Passed

The backend suite started real Python workers against a real PostgreSQL database. It completed separate dummy tasks with two workers and processed one valid image alongside one corrupt image, producing PARTIAL_SUCCESS. These tests used native execution, not Docker workloads.

Run the checks again

From the project root:

```
.venv\Scripts\python.exe -m unittest discover -s workloads/tests -v
.venv\Scripts\python.exe -m unittest discover -s worker/tests -v
cd control-plane
$env:TEST_DATABASE_URL='YOUR_TEST_DATABASE_CONNECTION_STRING'
npm test
```

The backend integration suite creates and removes a uniquely named test schema. Its database role needs schema creation permissions. Without TEST_DATABASE_URL, the integration suite is skipped, so a green unit-only run is not equivalent to full verification. TEST_PYTHON can select the Python interpreter; otherwise the suite uses the project's .venv.

In `frontend`, run `npm run build` and `npm run lint`. After changing APIs, use the complete integration suite and a browser acceptance test before demo day.

13 Troubleshooting and current limits

Common operating problems

Symptom	Check and action
Dashboard cannot connect	API health, configured API/proxy URL and ADMIN_TOKEN; errors are now visible
Database authentication fails	DATABASE_URL credentials and port; the old local .env requires review
Worker cannot enroll	Matching ENROLLMENT_TOKEN, API address and network reachability
Docker client fails	Start Docker and confirm both client and server appear in docker version
Image not found on worker	Build both configured workload images before starting Docker-mode agent
Job remains queued	Worker online status, capability list and required RAM
Worker reports another agent is running	Use a unique DATA_DIR; do not share credentials between active agents
Image task returns partial success	Download the JSON errors and identify corrupt or unsupported files
Signed transfer fails	Check advertised storage address, time synchronization and attempt validity
Job seems slow to start	Idle workers back off to 30 seconds between polls; verify they are registered
API stops unexpectedly	Inspect API/database logs; restart after correcting the actual error

Validation environment notes

Docker Desktop in this session failed while initializing its inference listener. A separate local PostgreSQL instance under `.test-data/postgres`, on localhost port 55433, was used for validation. This test fixture is not the normal Compose database and should not be assumed to be running when the project is reopened.

The local test API, worker and frontend processes are not a permanent installation. Start the documented services again for your own session. Browser rendering/interaction was not verified: the in-app browser blocked the local page and a later connectivity check did not reach the dev server. The frontend build and lint are verified, but they are not a substitute for clicking through the UI on the demonstration machine.

Features not currently delivered

• Object detection with bounding boxes, YOLO, OCR, chat, image generation or model training.

• Splitting one inference/model across several computers or providing shared RAM.

• GPU scheduling, validated ARM deployment or automated worker installation.

• User accounts, roles beyond the shared admin/worker model, billing or public untrusted enrollment.

• Job cancellation, pause/resume, priority editing, manual reassignment or a delete UI.

• WebSockets; the dashboard uses polling.

• Automatic AWS infrastructure provisioning, production migrations or high-availability deployment.

• A dataset ZIP import, arbitrary external S3 URI submission or arbitrary user code execution.

• Measured whole-grid energy savings, demonstrated carbon neutrality or verified avoided manufacturing emissions.

Storage cleanup/lifecycle management, long-running production observability and large-scale load testing remain operational work. The local disk adapter should not be used across multiple independent API instances without shared storage. The standalone advanced scoring and environmental helpers are not integrated dashboard features.

14 Hackathon demonstration and next steps

A practical demonstration sequence

1. Start the controller and two workers before presenting. Verify the dashboard connection and check that the workers advertise the intended workload.

2. Explain the purpose: older computers can contribute useful CPU work rather than remaining idle or being discarded prematurely.

3. Submit a pipeline demo with enough chunks to distribute across both workers. Show the node states and completed-task counts.

4. Submit a small batch of genuine photographs. Download the classification JSON and explain the class/confidence fields.

5. Include a deliberately corrupt image in a separate test batch. Show that valid results survive and the job reports partial success.

6. Show the measured execution duration and estimated energy consumption. Explain the wattage assumption and its limits.

7. If demonstrating worker loss, prepare a sufficiently long task and allow for the lease timeout. Show reassignment/retry behavior without claiming instant recovery.

The default five-minute lease may be too long for a short presentation. A separate demo instance can use shorter lease, heartbeat and sweep settings, but validate that setup beforehand. Network transfers and task execution still need enough time to complete normally.

What you can accurately tell judges

"We built a CPU batch-compute grid for trusted retired computers. The controller splits jobs and workers pull compatible tasks. We demonstrated real image classification, result aggregation and partial-failure recovery, with automated checks for duplicate submissions, concurrency and expired leases. The dashboard reports estimated compute energy; physical measurements and production deployment are further validation steps."

Before calling it fully ready for your event

• Run the Compose stack or physical-worker setup on the exact demonstration machines.

• Click through login, submission, file selection, progress, errors and result download in the chosen browser.

• Confirm useful predictions on genuine images and record the machine specifications.

• Run a real network-disconnection test and record how long reassignment takes.

• If using AWS, verify bucket permissions, signed transfers, HTTPS and deployment restart recovery.

• Keep a small local dataset and the pipeline demo available as a fallback.

Key files for future work

The original `architecture_design.md` remains the architecture baseline. This guide and the current API contracts describe implementation decisions made since that baseline, including local storage, attempt tokens, individual image uploads, the actual FP32 model and the explicit native development path.

Start code changes in `control-plane/src/app.ts` for orchestration, `worker/agent.py` for worker lifecycle, `worker/task_executor.py` for execution, and `frontend/src/App.tsx` for the user flow. The Prisma schema owns persisted state. The integration tests are the best executable reference for how the components communicate.

Technical references

PostgreSQL task coordination is implemented in the application source and verified by the integration suite. Prisma transaction guidance: https://www.prisma.io/docs/orm/v6/prisma-client/queries/transactions

AWS SDK S3 operations and presigned URL examples: https://docs.aws.amazon.com/sdk-for-javascript/v3/developer-guide/javascript_s3_code_examples.html

These references explain the library mechanisms. The project's own test results, rather than library documentation, are the evidence for the verified behaviors described above.